>_ MCMP113

Introduction to Basic Algorithms and Programming

Week 9

One-Dimensional Arrays

Declaration, assignment and element access · many values under one name

```
> int[] a = { 3, 7, 1 };
> a[1] = 9;
> string.Join(", ", a)
3, 9, 1
> a.Length
3
> a[3]
IndexOutOfRange…
> _
```

Asst. Prof. Hasan Oğuz, PhD
Istanbul Okan University · Vocational School
2026–2027 Fall Semester

By the End of Today You Can …

Explain why

Say why an array beats ten separate variables, and when you need one.

Draw the boxes

Draw an array as numbered boxes: index from 0, last index Length − 1.

Declare & create

Write new int[n] and initializer lists; know the default values.

Access elements

Read and change a[i]; avoid IndexOutOfRangeException.

Traverse

Process every element with for and foreach; read N values; print an array.

Copy correctly

Tell a second name (b = a) from a real copy (Clone or a loop).

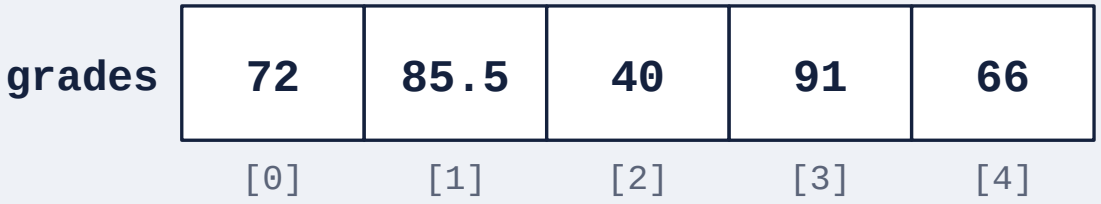
A Question Our Loops Cannot Answer Yet

Weeks 6–8: one variable, overwritten

```
sum ← 0
FOR i ← 1 TO n
  READ x              // x is overwritten
  sum ← sum + x
END FOR
average ← sum / n
PRINT how many x > average ???
```

Too late! When the average is known, x holds only the last grade. The other grades are lost.

Week 9: keep every value in an array



Average 70.9 → go through the boxes again: 72, 85.5 and 91 are above it → 3 of 5.

Array = many values of the same type under ONE name, each in its own numbered box.



PART 1

Why Arrays? The Memory Model

One name, many numbered boxes — and why the first box is number 0

Many Grades: Many Variables or One Array?

Five grades without an array

```
double g1 = double.Parse(Console.ReadLine());
double g2 = double.Parse(Console.ReadLine());
double g3 = double.Parse(Console.ReadLine());
double g4 = double.Parse(Console.ReadLine());
double g5 = double.Parse(Console.ReadLine());
double sum = g1 + g2 + g3 + g4 + g5;
// 100 grades: 100 names, 100 lines ...
```

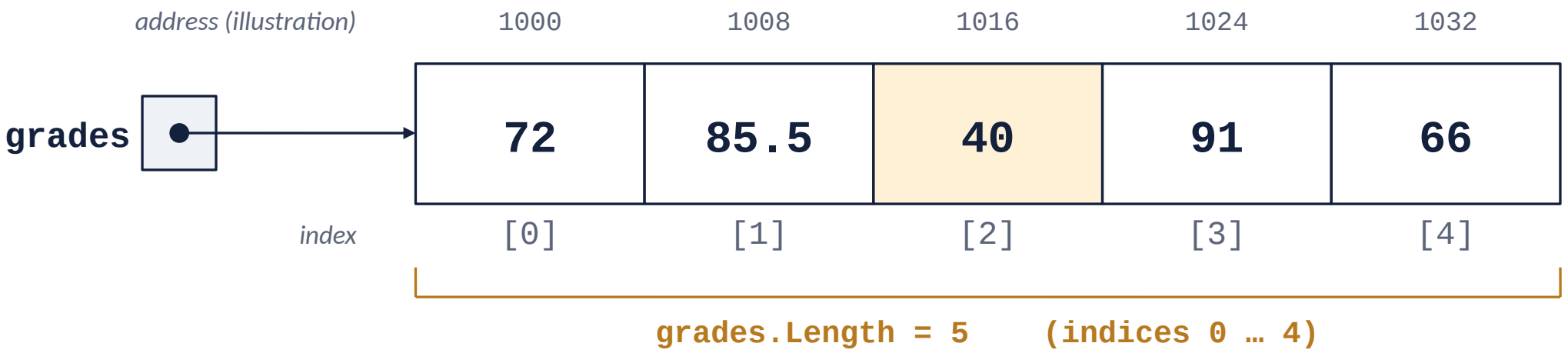
n names · a loop cannot help

Five grades with an array

```
double[] g = new double[5];
double sum = 0;
for (int i = 0; i < g.Length; i++)
{
    g[i] = double.Parse(Console.ReadLine());
    sum += g[i];
}
```

1 name + index i · one loop, any size

An Array Is a Row of Numbered Boxes



Element

One box. grades[2] is the element with index 2; its value is 40.

Index

Position = number of boxes to skip from the start. First index 0, last Length − 1.

Length & type

Fixed size, set when the array is created. All elements have the same type.



PART 2

Declaring, Creating, Accessing

new, default values, initializer lists, a[i] — and the exception you will meet

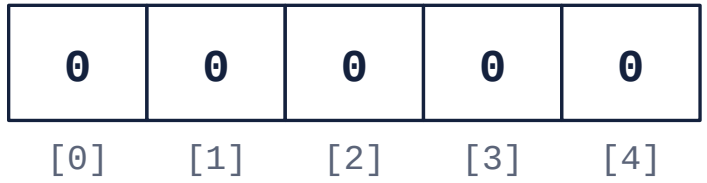
>_ DECLARATION

Anatomy of int[] scores = new int[5];

```
int[]   scores   =   new   int[5]   ;
```

- `int[]` — type: array of int
- `scores` — name
- `new` — create boxes
- `int[5]` — element type + size

Result in memory



Size known only at run time

09_02_GradeStats

```
Console.Write("How many students? ");
int n = int.Parse(Console.ReadLine());
double[] grades = new double[n];
```

Two steps are also fine

```
int[] scores;          // declared, no array yet
scores = new int[5];   // now 5 boxes exist
```

Not C/C++ style: int scores[5]; does not compile in C#. The size belongs to new, the brackets to the type.

new Fills Every Box with a Default Value

Element type	Default value	Example
int, long, short, byte	`0`	`new int[3] → 0, 0, 0`
double, float	`0.0 (prints 0)`	`new double[2] → 0, 0`
decimal	`0`	`new decimal[2] → 0, 0`
bool	`false`	`new bool[2] → False, False`
char	`'\0' (code 0)`	`new char[2] → '\0', '\0'`
string	`null`	`new string[2] → null, null`

09_01_ArrayBasics (excerpt)

```
double[] prices = new double[3];
bool[] present = new bool[3];
string[] names = new string[3];
char[] letters = new char[3];
```

Console

```
double: 0, bool: False
string is null: True, char code: 0
```



null ≠ "". null means "no string yet"; "" is a string with 0 characters. Fill string arrays before you use the elements.

Known Values? Write Them Directly

Short form (declaration only)

```
int[] a = { 3, 7, 1 };
```

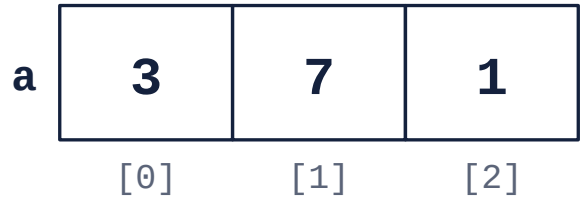
With new and the type

```
int[] a = new int[] { 3, 7, 1 };
```

Type inferred: new[]

```
var a = new[] { 3, 7, 1 };   // int[]
```

All three create



`a.Length = 3`

```
string[] days = { "Mon", "Tue", "Wed" };
char[] vowels = { 'a', 'e', 'i', 'o', 'u' };
```

The size is fixed. There is no "add one more box". a = new int[] { 9, 9 }; creates a NEW 2-element array and lets a refer to it; Length is read-only (a.Length = 4; does not compile).

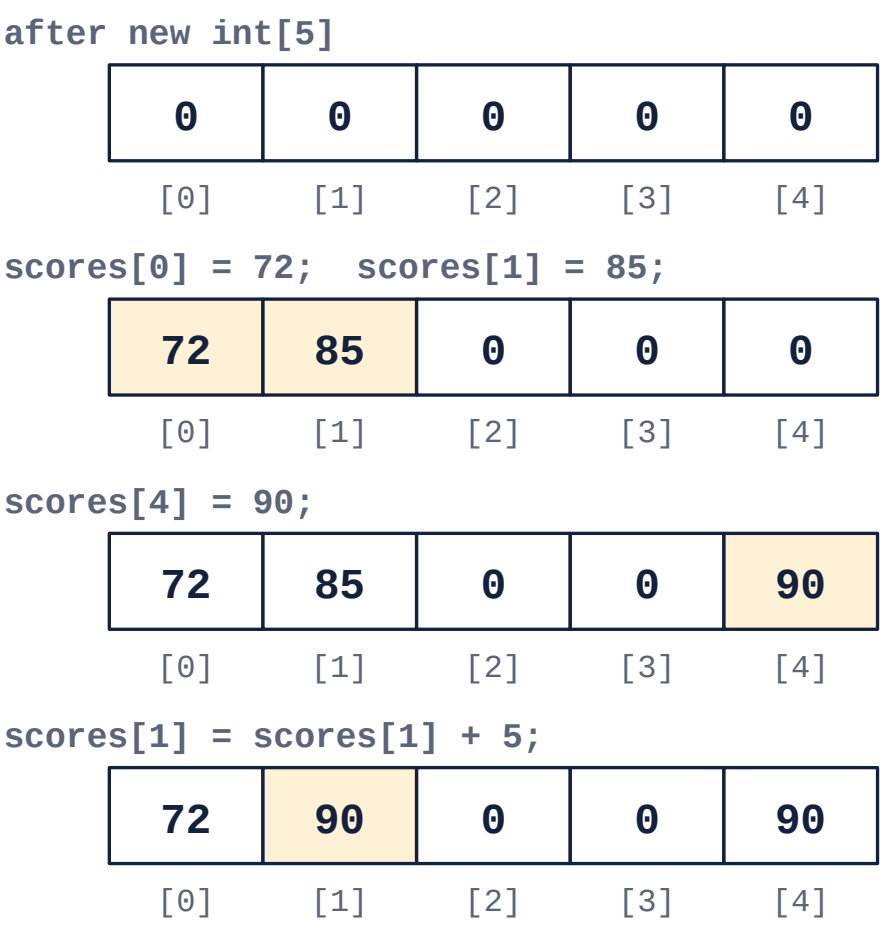
a[i] Reads a Box, a[i] = … Writes It

09_01_ArrayBasics (excerpt)

```
int[] scores = new int[5];
scores[0] = 72;
scores[1] = 85;
scores[4] = 90;
scores[1] = scores[1] + 5;
```

Console

```
Length: 5
After new int[5]: 0, 0, 0, 0, 0
After assignments: 72, 90, 0, 0, 90
First: 72, last: 90
```

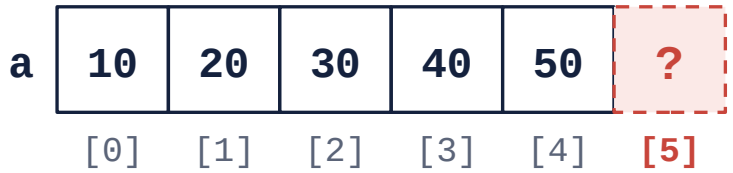


>_ RUN-TIME ERROR

IndexOutOfRangeException: the Box Does Not Exist

09_03_OutOfRange (excerpt)

```
int[] a = { 10, 20, 30, 40, 50 };
Console.Write("Index: ");
int index = int.Parse(Console.ReadLine());
if (index >= 0 && index < a.Length)
{
    Console.WriteLine($"Checked:   a[{index}] = {a[index]}");
}
else
{
    Console.WriteLine($"Checked:   {index} is not a valid index");
}
Console.WriteLine($"Unchecked: a[{index}] = {a[index]}");
```



valid: 0 … Length − 1 = 4

a[5], a[-1]: no such box

Run 1 (index 3)

```
Index: 3
Checked:   a[3] = 40
Unchecked: a[3] = 40
```

Run 2 (index 5) — the program stops at line 21

```
Valid indices: 0 .. 4
Index: 5
Checked:   5 is not a valid index
Unhandled exception. System.IndexOutOfRangeException: Index was outside the bounds of the array.
   at Program.<Main>$(String[] args) in …/09_03_OutOfRange/Program.cs:line 21
```

PREDICT · 4 min

Quiz 1: Indices and a Filling Loop

Program.cs

```
 1  int[] a = { 4, 8, 15, 16, 23 };
 2  Console.WriteLine(a[1] + a[3]);
 3  Console.WriteLine(a[a.Length - 1]);
 4  Console.WriteLine(a[a[0] - 2]);
 5  a[2] = a[0] * 2;
 6  Console.WriteLine(string.Join(",", a));
 7
 8  int[] b = new int[5];
 9  for (int i = 0; i < b.Length; i++)
10  {
11      b[i] = i * i;
12  }
13  Console.WriteLine(b[4] - b[2]);
14  int s = 0;
15  for (int i = 0; i < b.Length; i += 2)
16  {
17      s += b[i];
18  }
19  Console.WriteLine(s);
```

Your answer

1 _____

2 _____

3 _____

4 _____

5 _____

6 _____

ANSWER

Quiz 1: Answer

#	Output	Why
1	`24`	a[1] + a[3] = 8 + 16
2	`23`	a[a.Length − 1] = a[4], the last element
3	`15`	inner first: a[0] − 2 = 2 → a[2]
4	`4,8,8,16,23`	a[2] = a[0] * 2 = 8 overwrites 15
5	`12`	b = {0, 1, 4, 9, 16}; 16 − 4
6	`20`	i = 0, 2, 4: 0 + 4 + 16



09_13_PredictChecker

```
--- Quiz 1: indices ---
24
23
15
4,8,8,16,23
--- Quiz 2: filling with a loop
---
12
20
```

Real output (first part of 09_13).



PART 3

Traversing, Reading, Printing

for and foreach over every element · N values from the keyboard · four worked examples

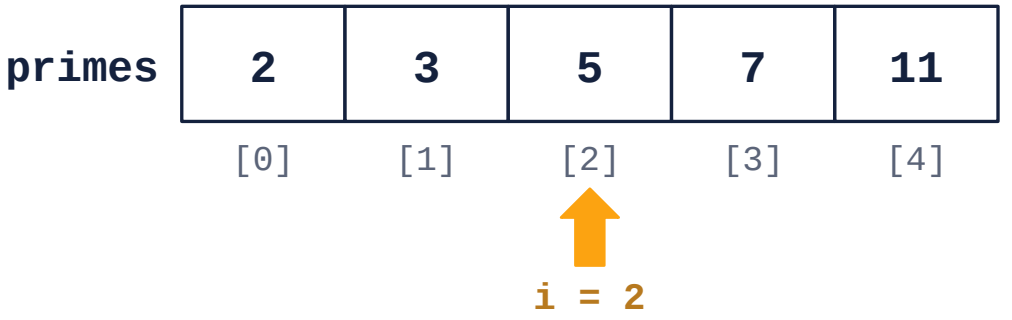
The Standard Loop: i from 0 While i < Length

09_01_ArrayBasics

```
int[] primes = { 2, 3, 5, 7, 11 };
for (int i = 0; i < primes.Length; i++)
{
    Console.WriteLine($"primes[{i}] = {primes[i]}");
}
```

Console

```
primes[0] = 2
primes[1] = 3
primes[2] = 5
primes[3] = 7
primes[4] = 11
```



i	i < 5 ?	action
0	true	print primes[0]
1 … 4	true	print primes[1] … [4]
5	false	loop ends

⚠ **`for (int i = 0; i <= a.Length; i++)`** → one pass too many → IndexOutOfRangeException.

foreach: Every Value, No Index

09_01_ArrayBasics

```
string[] days = new[] { "Mon", "Tue", "Wed" };
foreach (string day in days)
{
    Console.Write(day + " ");
}
```

Console

```
Mon Tue Wed
```

Not allowed

```
foreach (int x in a)
{
    x = x * 2;      // compile error
}
```

	for	foreach
index available	yes: i	no
can change elements	yes: a[i] = …	no (read-only)
neighbours a[i − 1]	yes	no
out-of-range risk	yes	no
use it to …	fill, change, compare	read every value

dotnet build

```
error CS1656: Cannot assign to 'x'
because it is a 'foreach iteration
variable'
```

Reading N Values · Printing an Array

Reading (09_02_GradeStats)

```
double[] grades = new double[n];

// Pass 1: read and store every grade
for (int i = 0; i < n; i++)
{
    Console.Write($"Grade {i + 1}: ");
    grades[i] = double.Parse(Console.ReadLine());
```

Console

```
How many students? 5
Grade 1: 72
Grade 2: 85.5
…
```

Printing: three ways

09_01_ArrayBasics

```
Console.WriteLine(primes);
Console.WriteLine(string.Join(", ", primes));
for (int i = 0; i < primes.Length; i++)
{
    Console.Write($"{primes[i],4}");
}
```

Console

```
System.Int32[]
2, 3, 5, 7, 11
   2   3   5   7  11
```

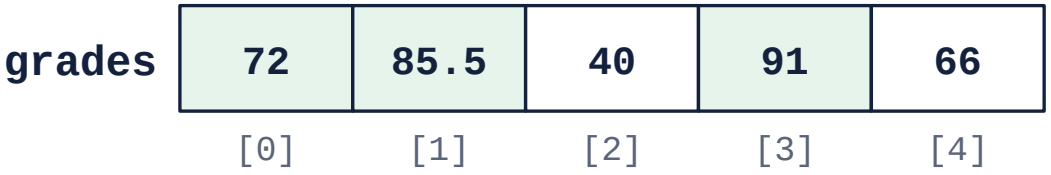
Grade Statistics: Average and Above-Average Count

Pseudocode

```
START
  READ n
  grades ← new array of n numbers
  FOR i ← 0 TO n − 1
    READ grades[i]
  END FOR
  sum ← 0
  FOR i ← 0 TO n − 1
    sum ← sum + grades[i]
  END FOR
  average ← sum / n
  count ← 0
  FOR i ← 0 TO n − 1
    IF grades[i] > average THEN count ← count + 1
  END FOR
  PRINT average, count
END
```

09_02_GradeStats

```
How many students? 5
Grade 1: 72
Grade 2: 85.5
Grade 3: 40
Grade 4: 91
Grade 5: 66
Grades: 72 | 85.5 | 40 | 91 | 66
Average: 70.90
Above average: 3 of 5
```



green = above 70.9

Grade Statistics in C#

Program.cs (1/2)

```
Console.Write("How many students? ");
int n = int.Parse(Console.ReadLine());

double[] grades = new double[n];

// Pass 1: read and store every grade
for (int i = 0; i < n; i++)
{
    Console.Write($"Grade {i + 1}: ");
    grades[i] = double.Parse(Console.ReadLine());
}
```

Program.cs (2/2)

```
// Pass 2: the sum and the average
double sum = 0;
for (int i = 0; i < grades.Length; i++)
{
    sum += grades[i];
}
double average = sum / grades.Length;
// Pass 3: count the grades above the average
int aboveCount = 0;
for (int i = 0; i < grades.Length; i++)
{
    if (grades[i] > average)
    {
        aboveCount++;
    }
}
Console.WriteLine($"Grades: {string.Join(" | ", grades)}");
Console.WriteLine($"Average: {average:F2}");
Console.WriteLine($"Above average: {aboveCount} of {n}");
```

>_ WORKED EXAMPLE 2

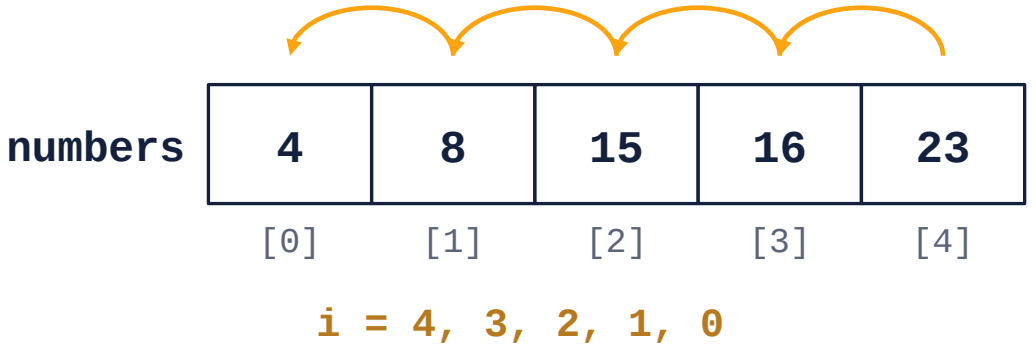
Printing in Reverse: Store First, Then Walk Back

09_04_ReversePrint

```
Console.Write("How many numbers? ");
int n = int.Parse(Console.ReadLine());
int[] numbers = new int[n];

for (int i = 0; i < n; i++)
{
    Console.Write($"Number {i + 1}: ");
    numbers[i] = int.Parse(Console.ReadLine());
}

Console.Write("Reversed:");
for (int i = numbers.Length - 1; i >= 0; i--)
{
    Console.Write($" {numbers[i]}");
}
Console.WriteLine();
```



Console

```
How many numbers? 5
Number 1: 4
Number 2: 8
Number 3: 15
Number 4: 16
Number 5: 23
Reversed: 23 16 15 8 4
```

>_ WORKED EXAMPLE 3

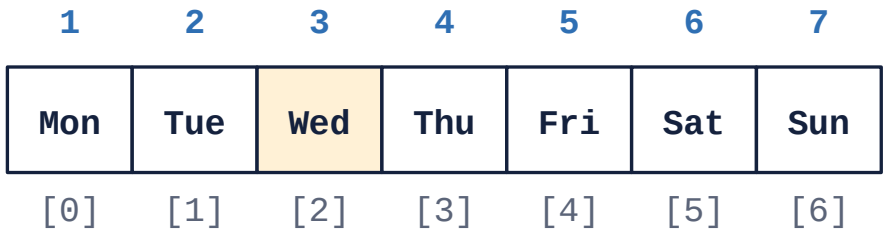
A Lookup Table Replaces a switch

09_05_DayNames

```
string[] dayNames = { "Monday", "Tuesday", "Wednesday", "Thursday",
                      "Friday", "Saturday", "Sunday" };
int[] lessonHours = { 4, 6, 2, 6, 3, 0, 0 };

Console.Write("Day number (1-7): ");
int day = int.Parse(Console.ReadLine());

if (day < 1 || day > dayNames.Length)
{
    Console.WriteLine("There is no such day.");
}
else
{
    int index = day - 1;
    string name = dayNames[index];
    Console.WriteLine($"{name}: {lessonHours[index]} lesson hours");
}
```



index = day − 1: day 3 → dayNames[2]

Run 1

```
Day number (1-7): 3
Wednesday: 2 lesson hours
```

Run 2

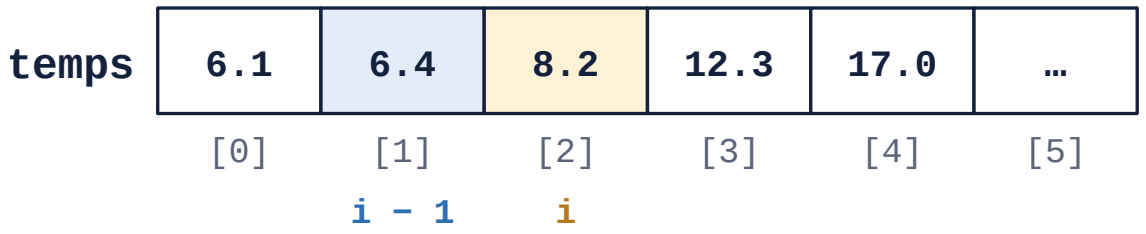
```
Day number (1-7): 8
There is no such day.
```

>_ WORKED EXAMPLE 4

Monthly Temperatures: Neighbours a[i – 1] and a[i]

09_06_MonthlyTemperatures (excerpt)

```
Console.WriteLine("Month   Temp   Change");
// January has no previous month: the loop starts at index 1
Console.WriteLine($"{months[0],-5}{temps[0],7:F1}{"-",9}");
for (int i = 1; i < temps.Length; i++)
{
    double change = temps[i] - temps[i - 1];
    Console.WriteLine($"{months[i],-5}{temps[i],7:F1}{change,9:F1}");
}
```



change = 8.2 – 6.4 = 1.8

Start at i = 1: temps[0 – 1] does not exist.

Console

```
Year average: 14.71 C
Month   Temp   Change
Jan      6.1        -
Feb      6.4      0.3
Mar      8.2      1.8
Apr     12.3      4.1
May     17.0      4.7
Jun     21.6      4.6
Jul     24.0      2.4
Aug     24.2      0.2
Sep     20.6     -3.6
Oct     16.2     -4.4
Nov     11.8     -4.4
Dec      8.1     -3.7
Warmer than average: 17.0 21.6 24.0
24.2 20.6 16.2
```



PART 4

More Types, Patterns and References

char and string · counting arrays · parallel arrays · copy or second name? · building from neighbours

Arrays of Any Type · a String Is Almost a char Array

```
double[]    double[] temps = { 6.1, 6.4 };

string[]    string[] names = { "Ayşe", "Can" };

char[]      char[] vowels = { 'a', 'e', 'i' };

bool[]      bool[] present = new bool[30];
```

Strings are read-only

```
string s = "cat";
s[0] = 'b';          // compile error
```

string text = "Hello";



Expression	Value
text.Length	5
text[0]	'H' (a char)
text[text.Length − 1]	'o'
text[5]	IndexOutOfRangeException

```
dotnet build

error CS0200: Property or indexer
'string.this[int]' cannot be assigned
to -- it is read only
```

>_ WORKED EXAMPLE 5

Vowel Counter: Indexing a String

09_09_VowelCounter (lines 4–25)

```
char[] vowels = { 'a', 'e', 'i', 'o', 'u' };
int[] vowelCounts = new int[vowels.Length];
Console.Write("Text: ");
string text = Console.ReadLine();
int letters = 0;
for (int i = 0; i < text.Length; i++)
{
    char c = char.ToLower(text[i]);
    if (char.IsLetter(c))
    {
        letters++;
    }
    for (int v = 0; v < vowels.Length; v++)
    {
        if (c == vowels[v])
        {
            vowelCounts[v]++;
        }
    }
}
```

lines 27–33 (printing lines 34–41 not shown)

```
int totalVowels = 0;
for (int v = 0; v < vowels.Length; v++)
{
    Console.Write($"{vowels[v]}:{vowelCounts[v]}  ");
    totalVowels += vowelCounts[v];
}
Console.WriteLine();
```

Console

```
Text: Programming is fun!
a:1  e:0  i:2  o:1  u:1
Letters: 16, vowels: 5
Consonants: 11
First: 'P', last: '!'
```

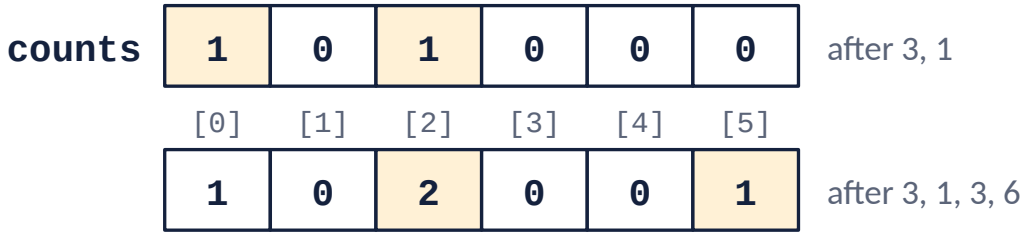
Dice Frequencies: counts[face – 1]++

09_07_DiceHistogram (excerpt)

```
Random rng = new Random(2026);
int[] counts = new int[6];

for (int r = 0; r < rolls; r++)
{
    int face = rng.Next(1, 7);
    counts[face - 1]++;
```

example rolls 3, 1, 3, 6: counts after two and after four rolls



60 rolls

```
How many rolls? 60
Face  Count  Share  Histogram
   1     16   26.7%  **************************
   2     12   20.0%  ********************
   3      3    5.0%  *****
   4      9   15.0%  ***************
   5      8   13.3%  *************
   6     12   20.0%  ********************
```

6000 rolls (header shortened: * = 1 %)

```
How many rolls? 6000
Face  Count  Share  Histogram
   1   1037   17.3%  *****************
   2    967   16.1%  ****************
   3    981   16.4%  ****************
   4   1036   17.3%  *****************
   5    971   16.2%  ****************
   6   1008   16.8%  *****************
```

Parallel Arrays: One Index, One Student

	[0]	[1]	[2]	[3]	[4]
names	Ayşe	Mehmet	Elif	Can	Zeynep
midterms	78	45	92	60	55
finals	85	40	88	70	38
totals	82.2	42.0	89.6	66.0	44.8

same index = same student

```
for (int i = 0; i < names.Length; i++)
{
    totals[i] = 0.4 * midterms[i] + 0.6 * finals[i];
    classSum += totals[i];
}
```

09_08_StudentReport

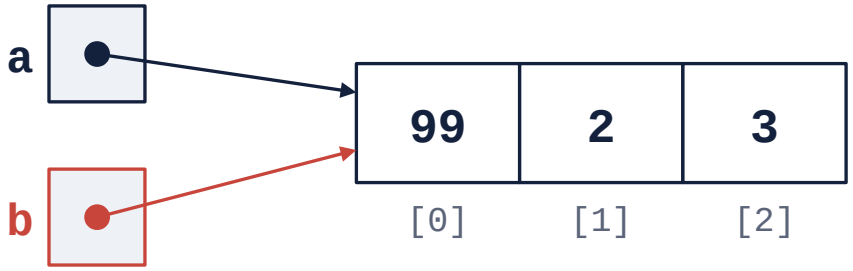
```
Name       Mid  Final   Total  Result
Ayşe        78     85    82.2  PASS
Mehmet      45     40    42.0  FAIL
Elif        92     88    89.6  PASS
Can         60     70    66.0  PASS
Zeynep      55     38    44.8  FAIL
Class average: 64.92
```



Keep them in step: same length, and every change (sort, delete) must be applied to all arrays at the same index.

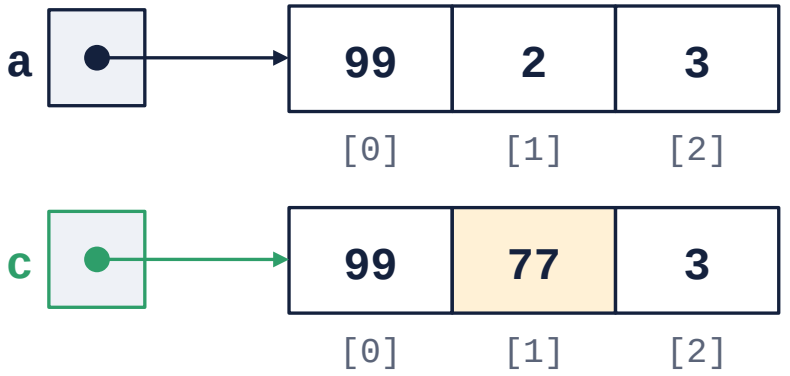
b = a Does Not Copy the Array

int[] b = a; → ONE array, two names



b[0] = 99 changes the only array → a[0] is 99 too.

int[] c = (int[])a.Clone(); → a new array



c[1] = 77 changes only the copy.

Statement	Copies …	After changing the new name
`int y = x;`	the value (value type)	x unchanged
`int[] b = a;`	the reference only	a changed too!
`int[] c = (int[])a.Clone();`	all elements (new array)	a unchanged
`copy loop into new int[a.Length]`	all elements (new array)	a unchanged

Proving It with a Program

09_10_CopyVsReference (excerpt)

```
int[] a = { 1, 2, 3 };
int[] b = a;
b[0] = 99;
Console.WriteLine($"   a = {string.Join(", ", a)}");
Console.WriteLine($"   b = {string.Join(", ", b)}");
int[] c = (int[])a.Clone();
c[1] = 77;
Console.WriteLine($"   a = {string.Join(", ", a)}");
Console.WriteLine($"   c = {string.Join(", ", c)}");
int[] d = new int[a.Length];
for (int i = 0; i < a.Length; i++)
{
    d[i] = a[i];
}
d[2] = 55;
Console.WriteLine($"   a = {string.Join(", ", a)}");
Console.WriteLine($"   d = {string.Join(", ", d)}");
Console.WriteLine($"a == b: {a == b},  a == c: {a == c}");
```

Console

```
1) int[] b = a;  b[0] = 99;
   a = 99, 2, 3
   b = 99, 2, 3
2) int[] c = (int[])a.Clone();  c[1] = 77;
   a = 99, 2, 3
   c = 99, 77, 3
3) copy loop into d;  d[2] = 55;
   a = 99, 2, 3
   d = 99, 2, 55
a == b: True,  a == c: False
int: x = 1, y = 99
```

Fibonacci and Running Totals

09_11_FibonacciArray (excerpt)

```
long[] fib = new long[n];
fib[0] = 0;
fib[1] = 1;
for (int i = 2; i < n; i++)
{
    fib[i] = fib[i - 1] + fib[i - 2];
}
```



fib[6] = fib[5] + fib[4] = 5 + 3

Console

```
How many Fibonacci numbers (2-93)? 10
0 1 1 2 3 5 8 13 21 34
fib[9] = 34
```

09_12_RunningTotal (excerpt)

```
total[0] = sales[0];
for (int i = 1; i < days.Length; i++)
{
    total[i] = total[i - 1] + sales[i];
}
```

Output after the 7 inputs (1250 … 1500)

```
Day       Sales      Total
Mon    1250.00    1250.00
Tue     980.50    2230.50
Wed    1430.00    3660.50
Thu       0.00    3660.50
Fri    2100.75    5761.25
Sat    3200.00    8961.25
Sun    1500.00   10461.25
Week total: 10461.25 TL
```

>_ COMMON MISTAKES

Six Array Mistakes and How to Avoid Them

`i <= a.Length`

Loop one pass too far. Use i < a.Length.

`a[a.Length]`

Not the last element: that is a[a.Length − 1].

`Console.WriteLine(a)`

Prints System.Int32[]. Use string.Join or a loop.

`int[] b = a;  as a copy`

Second name, same array. Use Clone() or a loop.

`user number = index`

Day 1..7 is index 0..6: subtract 1 (and check the range).

`int a[5];`

C/C++ syntax. In C#: int[] a = new int[5];

PREDICT · 5 min

Quiz 2: Names, Neighbours and Counters

Program.cs (part 1)

```
int[] x = { 1, 2, 3 };
int[] y = x;
y[0] = 10;
int[] z = (int[])x.Clone();
z[1] = 20;
x[2] = 30;
Console.WriteLine($"{x[0]} {x[1]} {x[2]}");
Console.WriteLine($"{y[0]} {y[1]} {y[2]}");
Console.WriteLine($"{z[0]} {z[1]} {z[2]}");
```

(part 2, continues)

```
int[] p = { 1, 2, 3, 4, 5 };
for (int i = 1; i < p.Length; i++)
{
    p[i] = p[i - 1] + p[i];
}
Console.WriteLine(string.Join(" ", p));
int[] data = { 2, 0, 2, 3 };
int[] c = new int[4];
foreach (int v in data)
{
    c[v]++;
}
Console.WriteLine(string.Join(" ", c));
```

Your answer

1 _____

2 _____

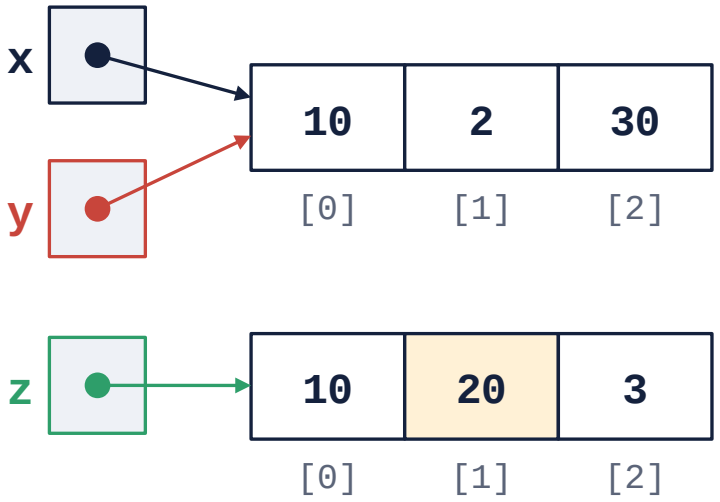
3 _____

4 _____

5 _____

ANSWER

Quiz 2: Answer



x and y: one array · z: a clone

#	Output	Why
1	`10 2 30`	x: y[0] = 10 and x[2] = 30 hit the same array
2	`10 2 30`	y is x
3	`10 20 3`	clone made after y[0] = 10; only z[1] changed
4	`1 3 6 10 15`	a[i] += updated a[i − 1]: running total
5	`1 0 2 1`	values 2, 0, 2, 3 used as indices of c

09_13_PredictChecker (2nd part)

```
10 2 30
10 2 30
10 20 3
1 3 6 10 15
1 0 2 1
```

Exam method: draw boxes and arrows first, then execute line by line and update the boxes.

>_ IN-CLASS · PAIRS · 30 MIN

Your Turn: Six Array Exercises

A Sums

Read 5 integers into an array; print the total and the sum of the elements with even index.

B Above the first

Read 10 numbers; print every number that is larger than the first one.

C Palindrome

Read a word; print whether it reads the same backwards (use text[i]).

D Two dice

Roll two dice 1000 times; count the sums 2..12 in an int[11]; print a histogram.

E Seat booking

bool[] seats = new bool[10]; reserve seat numbers until 0 is typed; print free seats.

F Vector sum

Read two arrays of length n; build c[i] = a[i] + b[i] and print all three.

>_ HOMEWORK · DUE BEFORE WEEK 10

Practice at Home

On paper

- H1 · Trace: int[] a = new int[5]; a[i] = (i + 1) * (i + 1) for i = 0..4; then print from the last index to the first.
- H2 · Draw the boxes and arrows after: int[] p = {5, 6}; int[] q = p; q[1] = 0; int[] r = (int[])p.Clone(); r[0] = 9. What do p, q, r contain?
- H3 · Write the pseudocode of exercise B with arrays.

On the computer

- H4 · Temperatures: read 7 daily temperatures; print the average and the days (Mon…Sun) warmer than the average.
- H5 · Turkish vowels: extend 09_09 with ı, ö, ü (and İ → i).
- H6 · Week 10 preview: find the largest element of an array and its index (start with a[0], not 0).

```
> cd code/09_02_GradeStats
> dotnet run
```

>_ SUMMARY

Key Takeaways

01 An array stores many values of one type under one name, in numbered boxes.

02 Indices run from 0 to Length − 1; anything else → IndexOutOfRangeException.

03 new T[n] fills with defaults (0, false, '\0', null); { … } lists set values and size.

04 for (int i = 0; i < a.Length; i++) to read/write; foreach to only read.

05 Patterns: lookup table, counting array, parallel arrays, build from neighbours.

06 b = a is a second name; copy with Clone() or a loop. Print with string.Join.

NEXT WEEK

Array operations: searching, sorting, max/min



MCMP113
Course materials